FraudShield

Credit Card Fraud Detection Platform

Project & Codebase Technical Guide

This document describes the full-stack implementation: architecture, source layout, fraud scoring pipeline, API surface, database design, authentication, role-based access, and frontend modules. Use it to prepare for project demonstration and technical questioning.

Generated: 18 May 2026, 09:49 UTC

1. Project overview

FraudShield is a web application that ingests card transactions, scores them for fraud risk using a hybrid approach (deterministic rules, behavioral profiling, and machine learning), stores decisions with audit trails, and presents results through a role-based operations console.

Problem addressed

- Detect suspicious card payments in near real time.
- Combine explainable rule signals with ML probability for analyst review.
- Support multiple personas: cardholder, fraud analyst, and administrator.
- Provide dashboards, alerts, exports, and investigation workflows.

Repository layout

- backend/ - Flask REST API, SQLAlchemy models, ML training & inference, migrations.
- frontend/ - Next.js 14 (App Router) UI with Tailwind CSS and Chart.js.
- scripts/ - Utility scripts including this PDF generator.
- DEFENSE.md / README.md - Quick-start and demo notes (Markdown).

2. System architecture

Three-tier pattern: React/Next.js client, Flask JSON API, SQLite database. The client stores JWT in localStorage and sends Authorization: Bearer on API calls. A lightweight session cookie (fraudshield_session) allows Next.js middleware to gate /dashboard routes.

Request flow (transaction ingest)

- 1. Client POST /transactions/ingest with amount, location, merchant, etc.
- 2. JWT validated; cardholder role forced to own user_id.
- 3. evaluate_transaction() runs rules + behavior + ML.
- 4. Transaction row saved; FraudDecision row stores breakdown.
- 5. UserProfile updated (rolling average spend, top locations).
- 6. If risk_score >= 60: status flagged, email alert stub, notification row.
- 7. AuditLog entry written for traceability.

Startup behaviour

On backend start, seed_all() ensures demo users and seeds ~80 sample transactions if the DB is sparse.

3. Technology stack

Backend (requirements.txt)

- Flask - HTTP API and application factory (app/__init__.py).
- Flask-SQLAlchemy + Flask-Migrate - ORM and schema migrations.
- Flask-JWT-Extended - JWT access tokens (24h expiry in config.py).
- Flask-Bcrypt - Password hashing.
- Marshmallow - Request validation schemas (app/schemas.py).
- scikit-learn, pandas, imbalanced-learn, joblib - ML training and inference.
- flask-cors - Cross-origin requests from Next.js dev server.
- fpdf2 - PDF report generation.

Frontend (package.json)

- Next.js 14.2 - App Router, server/client components.
- React 18 - UI.
- TypeScript 5.5 - Type safety.
- Tailwind CSS 3.4 - Styling, dark/light themes (next-themes).
- Chart.js + react-chartjs-2 - Dashboard charts.
- framer-motion - Landing page animations.
- lucide-react - Icons.

4. Backend structure (Flask)

Application factory - backend/app/__init__.py

Creates Flask app, loads Config, registers extensions (db, jwt, bcrypt), CORS, all blueprints, /health, and seeds data.

Blueprints and responsibilities

- auth (/auth) - register, login, password reset OTP, verify-email, /me.
- transactions (/transactions) - ingest, list, feed, flagged, simulator, PATCH actions.
- dashboard (/dashboard) - KPIs, charts, heatmap, audit logs, fraud rules list.
- admin (/admin) - user management, toggle FraudRule.enabled, model retrain stub.
- alerts (/alerts) - in-app notifications and email log.
- reports (/reports) - CSV, JSON, PDF exports and seed endpoint.
- fraud (/fraud) - simulate scoring without persist; explain by transaction id.
- users (/users) - profile, password change, sessions, 2FA toggle (demo).

Key services (backend/app/services/)

- transaction_ingest.py - Central pipeline: score, persist, profile update, alerts.
- rbac.py - scope_transactions(), is_staff(), can_manage_transaction().
- seed_data.py - Demo users and synthetic transaction seeding.
- simulator.py - Background thread ingesting every N seconds.
- transaction_generator.py - Random realistic transaction payloads.
- audit.py - log_action() writes AuditLog rows.
- alerts.py - send_email_alert() creates Alert records (simulated email).

Configuration - backend/app/config.py

- SECRET_KEY, JWT_SECRET_KEY from environment (defaults for dev).
- DATABASE_URL default: sqlite:///fraud_detection.db
- JWT_ACCESS_TOKEN_EXPIRES: 24 hours

5. Fraud detection pipeline

Core function: evaluate_transaction() in backend/app/fraud/engine.py

A. Rule engine (fraud/rules.py)

- Amount exceeds $5,000 -> +20 points.
- Five or more transactions in 10 minutes -> +15 points.
- Location differs from user's most recent transaction -> +10 points.

B. Behavioral engine (fraud/behavior.py)

- Uses UserProfile: avg_spend, top_locations (comma-separated), tx_count.
- Amount > 2.5x rolling average -> +15.
- Location not in historical top locations -> +10.
- Large jump vs last transaction amount -> +5.

C. Machine learning (fraud/model.py)

- Loads backend/ml/artifacts/fraud_model.joblib if present (trained via ml/train_model.py).
- Features: amount, tx_frequency_10m, minutes_since_last.
- Outputs probability; ml_score = probability * 35 (capped in final sum).
- Fallback heuristic if model file missing.

D. Score fusion and labels

- final_score = min(100, rule_score + behavior_score + ml_score)
- Labels: critical >=80, high >=60, medium >=30, else low.
- Transaction.status: flagged if score >= 60, else approved.

E. Explainability (fraud/explain.py)

Builds feature contribution list for charts (approximation for transparency). Used by GET /fraud/explain/<id> and fraud lab simulate endpoint. Note: Admin FraudRule table toggles are UI/catalog; live scoring uses hard-coded Python rules unless extended.

6. Database design

SQLite via SQLAlchemy. Main entities in backend/app/models.py:

- User - email, role (user|analyst|admin), password hash, is_active, email_verified.
- Transaction - amount, location, merchant, card fields, risk_score, status, confidence.
- FraudDecision - 1:1 with Transaction; stores rule/behavior/ml scores and reasons text.
- UserProfile - 1:1 with User; avg_spend, top_locations, tx_count.
- FraudRule - name, enabled, priority (admin-managed catalog).
- Alert - email delivery log linked to transaction.
- FraudNotification - in-app alert feed items.
- AuditLog - actor_user_id, action, entity, entity_id, JSON details.
- PasswordResetToken, UserSession - auth support tables.

Migrations

Flask-Migrate under backend/migrations/. Run: python -m flask --app run db upgrade

7. API reference (summary)

Authentication

- POST /auth/register {email, password, full_name, role}
- POST /auth/login -> {access_token, role, user_id}
- GET /auth/me JWT required
- POST /auth/forgot-password, verify-otp, reset-password

Transactions

- POST /transactions/ingest - Score and store (RBAC on user_id).
- GET /transactions/list?page&filters - Paginated, scoped for cardholder.
- GET /transactions/flagged, /transactions/feed?after_id=
- POST /transactions/simulator/start|stop|tick - Staff roles.
- PATCH /transactions/<id>/action {action: flag|approve|safe|freeze_account}

Dashboard & admin

- GET /dashboard/overview, /trends, /fraud-vs-legit, /heatmap, /model-metrics, ...
- GET /dashboard/audit-logs - Staff only.
- GET /dashboard/rules - Admin only.
- GET/PATCH /admin/users, PATCH /admin/rules/<id>, POST /admin/models/retrain

Reports & fraud

- GET /reports/transactions.csv, /summary.json, /summary.pdf, /audit-export.json
- POST /fraud/simulate, GET /fraud/explain/<transaction_id>

8. Frontend structure (Next.js)

Public routes

- / - Landing page (product overview, screenshot).
- /login, /register - Auth forms; register supports user/analyst/admin.
- /forgot-password, /verify-otp, /reset-password - Password recovery flow.

Dashboard routes (protected)

- /dashboard - Operations overview, KPI cards, charts, live feed.
- /dashboard/monitoring - Transaction table with filters (analyst/admin).
- /dashboard/capture - Manual ingest + 30s simulator (staff for stream).
- /dashboard/transactions - Flagged investigation queue.
- /dashboard/fraud - Fraud lab manual simulation.
- /dashboard/explain - Per-transaction explainability view.
- /dashboard/analytics, /alerts, /reports - Staff analytics and exports.
- /dashboard/admin - User/rule management (admin).
- /dashboard/profile - Account settings.

Important frontend modules

- lib/api.ts - fetchWithAuth, 401 handling, redirect to login.
- lib/auth-session.ts - localStorage token + session cookie for middleware.
- lib/roles.ts - Navigation and RoleGuard permissions.
- middleware.ts - Blocks /dashboard without session cookie.
- app/dashboard/layout.tsx - Validates JWT via /auth/me before render.
- components/transaction-notification-provider.tsx - Polls feed, toasts, sounds.
- components/kpi-card.tsx, charts/*, app-shell.tsx, sidebar.tsx

9. Authentication and RBAC

JWT contents

identity = user id (string). additional_claims.role = user | analyst | admin.

Role capabilities

- Cardholder (user): Own transactions only; overview scoped; capture manual; explain own txs.
- Analyst: Global monitoring, flagged queue, fraud lab, reports, simulator, freeze account.
- Admin: All analyst features + /admin users, rules, retrain stub, dashboard/rules API.

Demo accounts (seed_data.py)

- ops@fraudshield.demo / DemoPass123! - admin
- analyst@fraudshield.demo / DemoPass123! - analyst
- user@fraudshield.demo / DemoPass123! - cardholder

10. How to run locally

Backend

```
cd backend
pip install -r requirements.txt
python -m flask --app run db upgrade
```

```
python run.py
```

API: http://127.0.0.1:5000

Frontend

```
cd frontend
npm install
echo NEXT_PUBLIC_API_BASE=http://127.0.0.1:5000 > .env.local
npm run dev
```

UI: http://localhost:3000

11. Likely examination questions & answers

Q: Why hybrid rules + ML instead of ML only?

A: Rules give transparent, auditable triggers; ML catches non-linear patterns. Fusion improves recall while keeping explainability for analysts and regulators.

Q: How is explainability implemented?

A: Feature contributions are derived from rule text, behavior reasons, and ML inputs in explain.py and shown in the Explainability and Fraud lab pages.

Q: How do you secure the API?

A: JWT on protected routes, bcrypt passwords, role checks in routes and rbac.py, cardholder data scoped to own user_id, session validation on dashboard load.

Q: What database and why SQLite?

A: SQLite for development and demonstration portability; SQLAlchemy allows swapping to PostgreSQL in production via DATABASE_URL.

Q: How would you deploy to production?

A: Containerize backend + frontend, use PostgreSQL, set strong SECRET_KEY/JWT_SECRET_KEY, HTTPS, restrict CORS, connect real payment webhook to /transactions/ingest, retrain model on production data, integrate real email/SMS alerts.

Q: Limitations of this prototype?

A: Simulated email, stub 2FA, FraudRule DB not wired to engine, illustrative model metrics, no horizontal scaling or message queue yet.

12. File index (quick reference)

- backend/run.py - Dev server entry.
- backend/app/__init__.py - App factory.
- backend/app/models.py - All ORM models.
- backend/app/fraud/engine.py - Scoring orchestration.
- backend/ml/train_model.py - Train fraud_model.joblib.
- frontend/app/dashboard/page.tsx - Main dashboard UI.
- frontend/lib/api.ts - API client.